When Does Planning Help?
A Two-Layer Evaluation for Autonomous Coding Agents

Charles Ding

March 2026

Abstract

Autonomous coding agents spend a large fraction of their token budget on navigation: searching repositories, opening irrelevant files, and backtracking after wrong turns. Planning is an intuitive solution, but evaluating plan quality is itself non-trivial. This report presents a two-layer evaluation for planning in coding agents. Layer 1 is a scalable navigation evaluation over 234 SWE-bench tasks, using file targeting as the cheapest grounded proxy for whether a plan is useful. Layer 2 consists of targeted diagnostic probes that test where that proxy breaks, including content-quality scoring, code-generation ablations, plan criticism, refinement loops, prompt and model comparisons, and a small synthetic feature-planning supplement. The core result is that planning improves file recall by 35 percentage points on average over a keyword baseline, with strongly diminishing returns as tasks become more architectural. However, navigation accuracy and implementation reasoning quality are only partially coupled: plans can reason well about a task even when they fail to locate the right files. This suggests that agents should consume plans as two separate signals: file lists as soft navigation suggestions and reasoning as durable strategic context.

1 Introduction

Autonomous coding agents do not fail only because they cannot write code; they also fail because they spend too much effort looking in the wrong places. In large repositories with hundreds or thousands of files, a typical bug fix touches only a handful of files. The agent must first locate those files before it can reason about what to change. This search process is often expensive: it involves reading irrelevant code, following misleading stack traces, exploring dead ends, and backtracking after discovering that an initial hypothesis was wrong.

Planning—generating a structured outline of which files to inspect and modify before writing code—is a natural intervention. If a plan can narrow the search space early, it can save substantial tokens and latency. But planning is not free. Generating a plan consumes model calls, and an incorrect plan can be worse than no plan if it anchors the agent to the wrong part of the codebase.

This project began from a broad question: *does planning help coding agents?* That question quickly became an evaluation-design problem: *what is the cheapest grounded proxy for whether a plan is useful?* Rather than evaluate planning in the abstract, I chose to center the evaluation on codebase navigation. Planning is most obviously valuable if it reduces navigation entropy—that is, if it helps an agent identify the right code regions before expensive execution begins.

The result is a two-layer evaluation framework:

- **Layer 1: Navigation Evaluation.** A scalable, automated evaluation over 234 SWE-bench tasks using file recall, precision, and search reduction as grounded proxies for navigation quality.
- **Layer 2: Diagnostic Probes.** Smaller targeted experiments designed to test where file targeting is an incomplete quality signal: content-quality evaluation, code-generation ablations, plan critics, refinement loops, prompt and model comparisons, and a small synthetic feature-task supplement.

The core finding is that planning helps navigation everywhere, but with sharply diminishing returns as tasks become more architectural. More interestingly, the diagnostic probes reveal that planning provides two distinct kinds of value: a brittle navigation signal and a more durable reasoning signal. The resulting implication for agent design is not simply "plan or do not plan," but rather "which parts of the plan should the agent trust, and at what stage of execution?"

2 Evaluation Design Philosophy

I approached this as an evaluation-design problem first, not a benchmarking exercise. The core question was:

What is the cheapest grounded proxy for whether a plan is useful to a coding agent?

2.1 Why File Targeting Is the Core Proxy

File targeting emerged as the most practical answer. For a coding agent, a plan is useful if it helps narrow the repository search space before expensive execution begins. Measuring whether the plan identifies the files that actually need to change has four advantages:

- **Cheap**: one planning call per task, with no code execution required.
- **Automated**: ground truth comes from merged SWE-bench patches.
- **Grounded**: the metric directly captures the navigation value proposition of planning.
- **Scalable**: the evaluation can run on hundreds of tasks without manual labeling.

This motivated Layer 1: a large-scale navigation evaluation using file recall, precision, and search reduction.

2.2 Why a Second Layer Was Necessary

A single proxy is never the full story. File targeting might miss two important phenomena:

1. A plan could find the right files but still provide poor implementation guidance.
2. A plan could fail at file targeting but still reason correctly about the nature of the task.

For that reason, I added Layer 2: a set of targeted probes designed to stress-test where the Layer 1 proxy succeeds and where it breaks. These probes ask:

- Does better file targeting actually improve downstream code generation?
- What value remains when file targeting is poor?
- Can we detect bad plans before execution?
- Can bad plans be improved through critique and refinement?
- How sensitive is planning quality to model and prompt choice?
- Does the framework generalize beyond bug-fix tasks to more open-ended feature tasks?

3 Method

3.1 Task Selection

Pilot (n=14). I first curated 14 tasks from SWE-bench (9 Verified, 5 full) drawn from 8 repositories: Django (4), SymPy (2), Pylint (2), Matplotlib (2), Astropy (1), scikit-learn (1), Sphinx (1), and xarray (1). Selection criteria required at least 2 files across at least 2 directories, with fix location not trivially derivable from the issue title.

Extended set (n=234). I then scaled to 234 tasks from SWE-bench Verified, applying the same criteria programmatically. Tasks were stratified into three complexity tiers based on the number of ground-truth changed files:

Tier	Ground-truth files	Pilot (n)	Extended (n)
Localized	2–3	5	142
Cross-module	4–6	5	54
Architectural	7+	4	38

Table 1: Task stratification by ground-truth file count.

3.2 Plan Generation

For each task, I called the Claude API with the issue description and the complete repository file tree. The prompt asked for a structured JSON plan containing `files_to_inspect`, `files_to_modify`, and `implementation_steps`. Plans were generated in a single pass with no iterative refinement. The model had access only to file paths and issue text, not file contents.

3.3 Baseline

As a non-LLM baseline, I extracted salient terms from the issue text (module names, file names, function names, technical terms) and matched them against repository file paths. Any file whose path matched a keyword was included in the baseline candidate set. This approximates the retrieval heuristic an agent might use before reading code.

3.4 Metrics

The primary Layer 1 metrics were:

- **File Recall**: fraction of ground-truth changed files identified by the plan.
- **File Precision**: fraction of plan-identified files that actually changed.
- **Search Reduction**: fraction of the repository the agent needs to inspect if it follows the plan.

Path matching used normalized file paths to account for format differences between plan output and SWE-bench ground truth.

4 Layer 1: Navigation Evaluation

4.1 Main Result: Planning Helps Navigation with Diminishing Returns

Across 234 tasks, plans achieved a mean file recall of 0.529 versus 0.178 for the keyword baseline, a gain of 35 percentage points. The improvement was statistically significant at every complexity tier.

Tier	n	Plan Recall	Baseline Recall	Delta	p-value
Localized (2–3 files)	142	0.622	0.208	+0.414	<0.0001
Cross-module (4–6 files)	54	0.480	0.129	+0.351	<0.0001
Architectural (7+ files)	38	0.254	0.138	+0.116	0.0007
Overall	234	0.529	0.178	+0.351	<0.0001

Table 2: Navigation evaluation: plan recall versus keyword baseline.

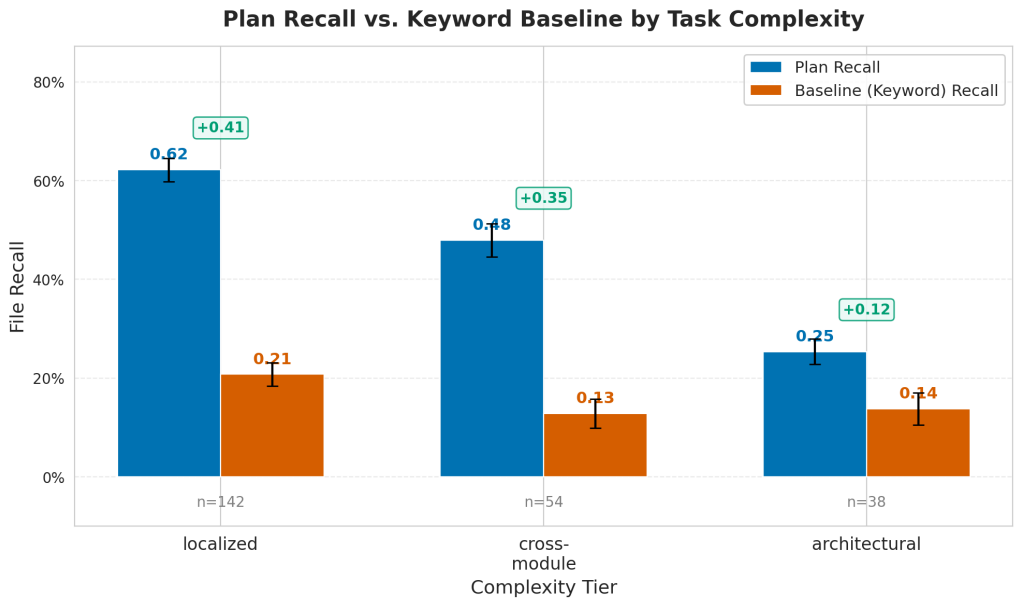

Figure 1: **Main navigation result.** Planning improves file recall across all three task tiers, with strongly diminishing returns as tasks become more architectural. This is the headline Layer 1 result and should appear early in the presentation.

The pilot study had suggested near-zero benefit on architectural tasks. At scale, the story changed: planning still helps on architectural tasks, but only modestly. The result is therefore not a binary crossover but a gradient. Planning is universally helpful for navigation, but its return declines as tasks involve more files and more diffuse dependencies.

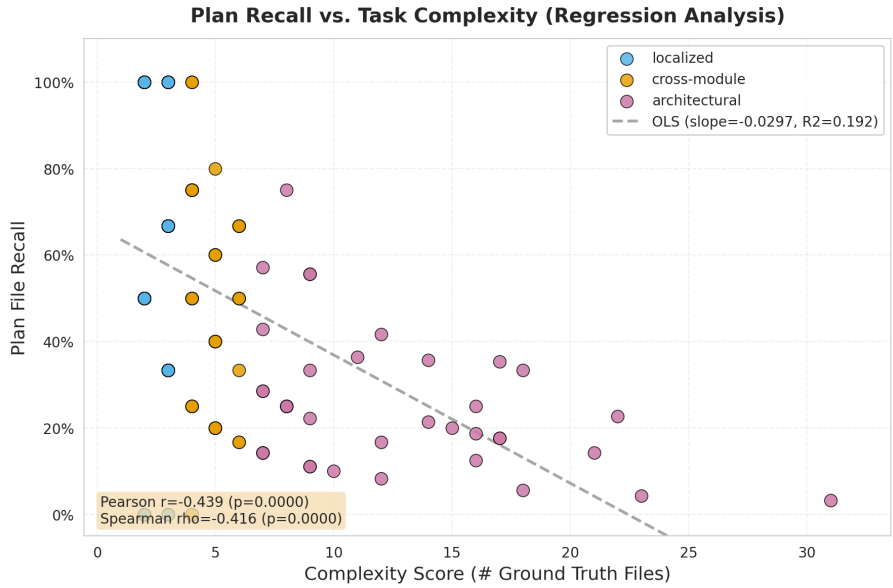

Figure 2: **Continuous complexity view.** File recall declines steadily as the number of ground-truth changed files increases. Use this plot to reinforce that the main finding is a gradient rather than a discrete threshold.

A linear regression of plan recall against task complexity yielded $r = -0.439$ ($p < 0.0001$), reinforcing that the decline is continuous rather than merely bucket-specific.

4.2 Where Static Plans Fail at the File Level

Aggregate recall hides an important structural story: which kinds of files are invisible to static plans?

Across 1,073 ground-truth files, I categorized each file by its relationship to the issue location and computed miss rates.

File Category	Miss Rate	n
Distant Module	100%	208
Sibling Package	100%	193
Shared Utility	61%	108
Documentation	50%	2
Config / `__init__.py`	44%	62
Same Package	27%	499

Table 3: Miss rates by structural file category.

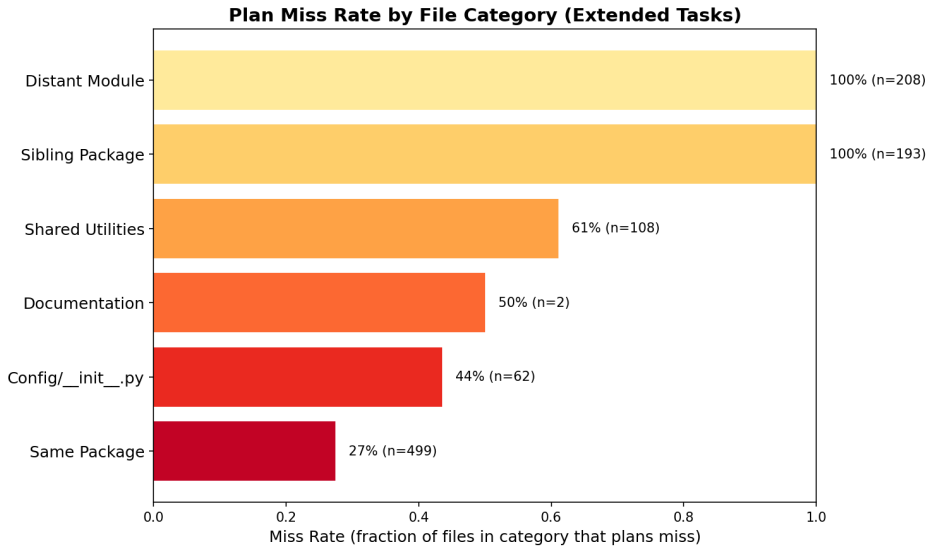

Figure 3: **Structural failure modes of static plans.** Plans succeed primarily on same-package files, miss shared utilities frequently, and fail almost completely on sibling and distant packages. This plot is especially useful for the "why plans fail" section of the presentation.

This suggests a structural ceiling on static planning. Same-package files are often visible from naming, locality, and module structure. Shared utilities are harder because they are rarely mentioned in issue text but often contain the idiomatic fix. Sibling and distant modules are often connected only by runtime behavior, not by static structure in the repository tree. Their 100% miss rate is not just a prompt-quality issue; it reflects information that is genuinely unavailable to a static planner

that only sees the file tree.

5 Layer 2: Diagnostic Probes

5.1 Plan Content Quality: Navigation and Reasoning Are Distinct

File recall is a good navigation metric, but it does not fully capture whether a plan is useful. A plan also contains implementation steps, assumptions, and risks. To test whether navigation quality and implementation guidance were aligned, I used Claude Opus as a judge to score the 14 pilot plans on four content dimensions: step coverage, step granularity, approach alignment, and implementation usefulness.

Tier	n	File Recall	Content Composite	Assumptions / Risks
Localized	5	0.667	2.15	2.60 / 2.60
Cross-module	5	0.577	2.00	2.60 / 2.40
Architectural	4	0.192	2.06	3.00 / 3.00

Table 4: Navigation quality versus content-quality scores across the pilot set.

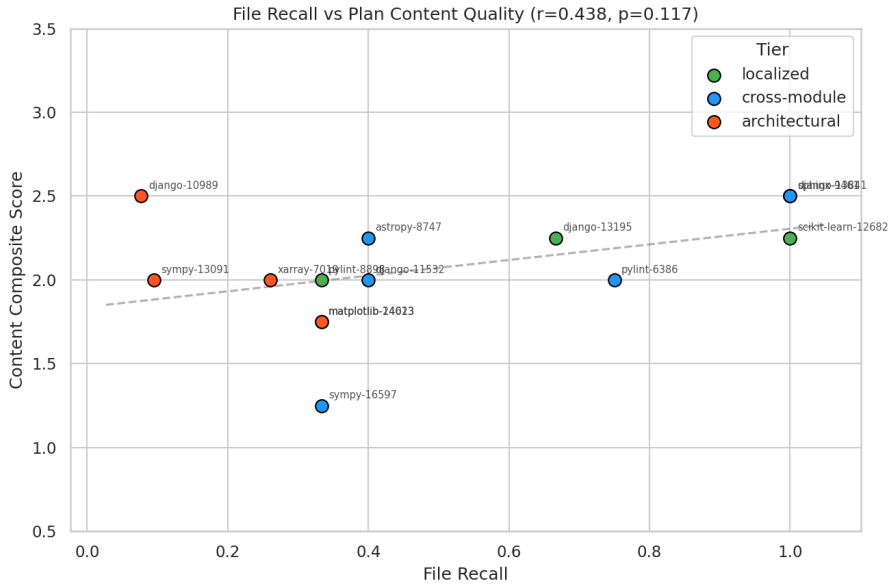

Figure 4: **Navigation versus reasoning.** This is the most important Layer 2 plot. File recall varies widely, but content quality is comparatively stable. Label representative disagreement cases such as `django-10989` (poor navigation, strong reasoning) and `sympy-16597` (poor on both).

The most important result is that navigation accuracy and reasoning quality are only partially coupled. Content scores remained flat across tiers even as file recall dropped sharply. Step coverage was the only dimension significantly correlated with file recall ($r = 0.583$, $p = 0.029$). Granularity and approach alignment were not strongly correlated.

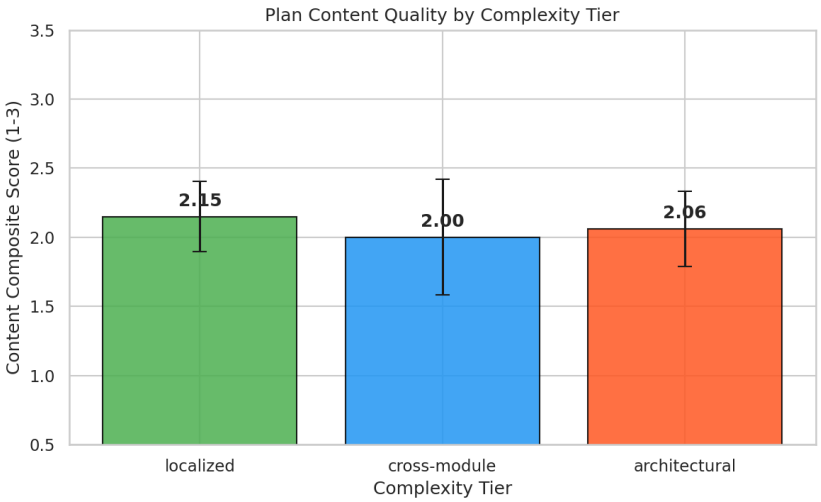

Figure 5: **Content quality by tier.** Use this plot to emphasize that reasoning quality remains roughly flat even as navigation quality deteriorates with complexity.

The clearest example is `django-10989`: file recall was only 0.08, but content quality was high. The plan correctly diagnosed a Unicode ellipsis problem and proposed the right strategy—replace problematic characters with ASCII equivalents—but failed to identify the full set of affected files. This means that even when a plan is weak as a file list, it may still be useful as strategic context.

5.2 Code Generation: Plans Help Reading More Than Writing

I next asked whether better plans actually improved downstream code generation.

This part of the evaluation used a deliberately weak setup: the model was asked to generate diffs without reading file contents. This is not how real agents operate, but it is useful as a probe of whether plans help or hurt when consumed naively.

On 50 tasks, plan quality correlated only moderately with downstream code-generation recall ($r = 0.356$). More strikingly, in direct plan-vs-no-plan comparisons over 40 tasks, the no-plan condition won 12 times, the plan condition won only 3 times, and the remainder were ties.

Note that we calculate win-rate through the following fashion. For both plan and no-plan settings, we generate two diffs on each of the tasks, extracting modified files paths from both settings. Then, we compare those paths against the ground-truth files from SWE-bench and compute the overall file recall. Then, the condition with the higher file recall "wins."

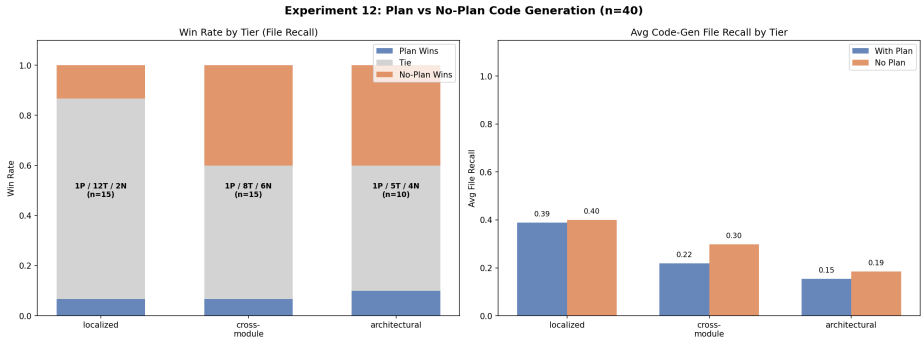

Figure 6: **Plan-guided versus unplanned code generation.** Plans often hurt blind diff generation because they anchor the model to brittle file lists. This plot should be used with an explicit caveat: code generation here did *not* have access to source code contents.

The interpretation is not that planning is harmful overall. Rather, plans appear to cause *informational anchoring* when fed directly into code generation. The file list narrows the model's attention too early and too rigidly. Plans are therefore most useful at the **reading stage**: they help decide what files to open. Once file contents are available, code should be generated from what the agent sees, not from what the plan predicted.

5.3 Plan Critic

I evaluated an LLM-based plan critic to see whether bad plans could be detected before execution. Binary verdicts were not useful: the critic flagged essentially everything. However, quantitative critic signals worked better. A composite score threshold identified the worst plans with high specificity.

This suggests that the right use of critics is not binary "accept/reject" gating but softer risk estimation. A critic can detect likely failure modes such as underestimated scope, missing shared utilities, or abstraction-level mismatch.

5.4 Refinement Loop

I also tested whether critique-revise loops improve bad plans. A single refinement pass improved file recall by about 9 percentage points on average across the worst plans, but a second pass often regressed.

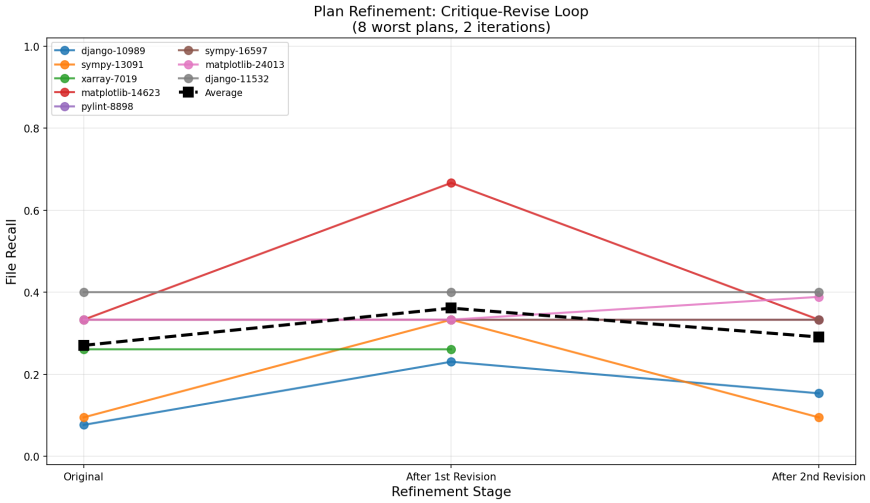

Figure 7: **Refinement dynamics.** One critique-revise pass often helps, but a second pass causes oscillation rather than convergence. This plot is useful as supporting evidence for a "single refinement pass" design recommendation.

The refinement experiment suggests that bad plans can often be improved, but repeated self-revision is unstable. A one-pass critic-refine loop is therefore more attractive than unrestricted iterative refinement.

5.5 Model and Prompt Sensitivity

I compared prompt variants and model choices on the pilot tasks. Prompting mattered most on cross-module tasks, where dependency-tracing prompts improved recall relative to minimal prompts. Model choice mattered less on routine tasks: Haiku was often competitive with Sonnet at much lower cost.

However, on the hardest architectural task (`matplotlib-24013`), Opus dramatically outperformed both Haiku and Sonnet.

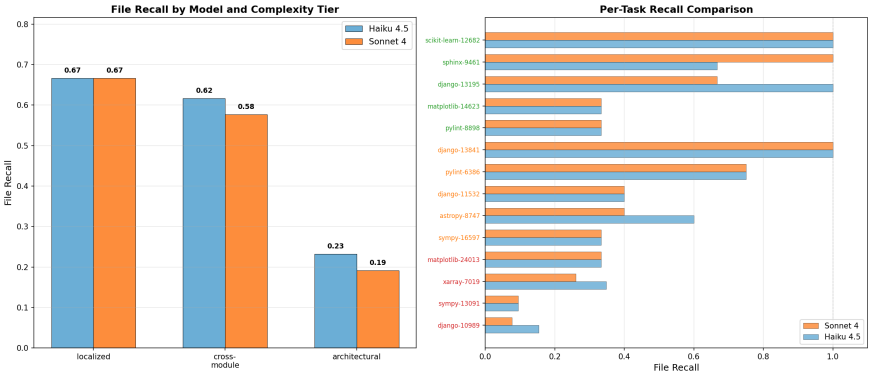

Figure 8: **Routine planning can often use cheaper models.** Haiku is competitive with Sonnet on the pilot set and is therefore attractive for always-on planning. Use this as a practical cost argument rather than a headline result.

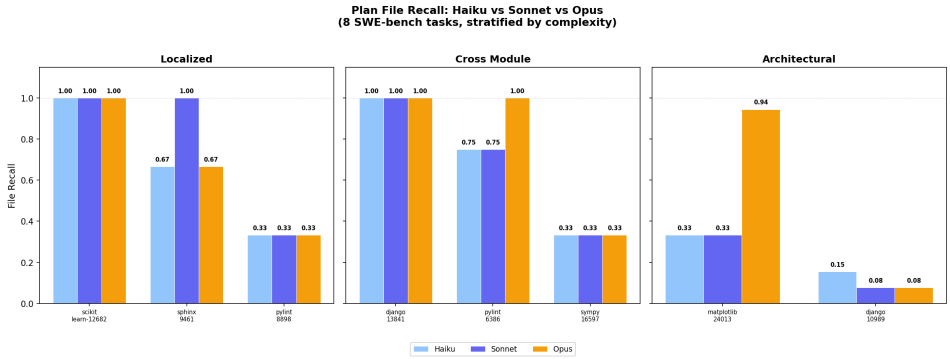

Figure 9: **Premium capability matters on the hardest tasks.** The standout case is `matplotlib-24013`, where Opus reaches 0.94 file recall versus 0.33 for cheaper models. This is the strongest visual argument for selective premium-model use.

The practical implication is a model cascade: use cheaper models for routine planning and route only the hardest tasks to premium models.

5.6 Generalization Beyond Bug Fixes: Synthetic Feature Tasks

Because SWE-bench is primarily bug-fix oriented, I added a small synthetic feature-task supplement to test whether the evaluation framework could transfer to more open-ended implementation tasks. This supplement is intentionally small: it is not a second benchmark, but rather an external-validity check on whether the same evaluation philosophy applies beyond bug localization.

I constructed a single synthetic repository, `dataflow/`, a CLI data-processing tool with roughly 30 files organized across multiple subpackages (`commands`, `formats`, `core`, `cache`, `utils`, `config`, and `tests`). Each file was implemented as a realistic stub: imports, class or function signatures, docstrings, and `NotImplementedError` bodies. The goal was not to create a fully runnable codebase, but to provide enough structure for a planner to infer module roles from file names and organization.

I then defined three feature tasks against this repository:

- **Task A: Add YAML Output Format.** Add a `-format yaml` option, create `yaml_output.py`, wire it into the output pipeline, add tests, and add `pyyaml` to requirements.
- **Task B: Add Redis Caching.** Add optional Redis-backed caching for pipeline results, configurable TTL, graceful fallback when Redis is unavailable, mock-based tests, and `redis` as a dependency.
- **Task C: Add `-dry-run` Flag.** Propagate a dry-run flag through argument parsing, commands, pipeline logic, and file I/O so writes are skipped and a summary is shown; add tests.

For these feature tasks, file recall is the wrong metric: unlike SWE-bench bug fixes, the correct solution often involves creating new files and touching multiple implementation surfaces that are not reducible to a fixed ground-truth changed-file set. Instead, I used **surface coverage**: whether the plan identified the key implementation surfaces required by the feature. Each task had a checklist of required surfaces labeled by category (`core_logic`, `interface`, `tests`, `config`) and by change type

(`create` vs. `edit`). A plan received partial or full credit depending on how completely it covered those required surfaces.

Dimension	Bug-Fix (SWE-bench)	Feature (Synthetic)
Evaluation substrate	234 real bug-fix tasks	1 synthetic repo, 3 feature tasks
Primary plan value	Navigation / file targeting	Decomposition / surface coverage
Main metric	File recall: 0.53	Surface coverage: 0.89
What plans get right	Same-package files	Core logic, tests
Main weak spot	Shared utilities, distant modules	Interface / integration surfaces

Table 5: Small generalization check from bug-fix planning to synthetic feature planning.

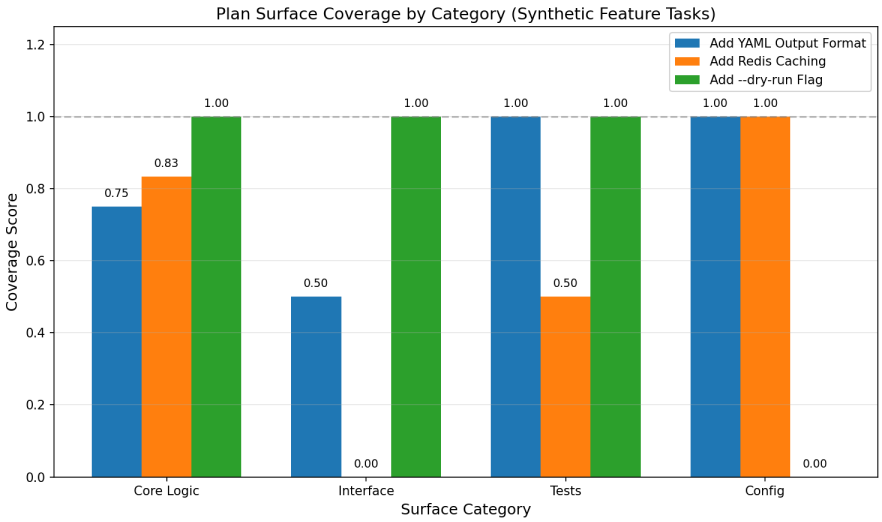

Figure 10: **Synthetic feature-task supplement.** Each colored series corresponds to one of three feature tasks in the `dataflow` repository, scored by coverage of required implementation surfaces (`core_logic`, `interface`, `tests`, `config`). This should be interpreted as a small external-validity check, not a co-equal benchmark.

The main pattern is that plans performed substantially better on these feature tasks than on bug-fix navigation. This is intuitive: feature requests often have more predictable implementation structure (new capability, integration point, tests, dependency updates), whereas bug fixes depend more heavily on latent dependencies and historical code organization. The weakest category was `interface`: plans were generally better at identifying core logic and tests than at consistently surfacing the integration points where a new feature must be exposed or wired into the system.

I therefore interpret this supplement not as evidence that feature planning is "solved," but as evidence that the evaluation framework transfers: for bug-fix tasks, planning is best evaluated as navigation; for feature tasks, planning is better evaluated as coverage of the required implementation surfaces.

6 Synthesis: What the Two Layers Show Together

Layer 1 establishes a scalable, grounded result: planning reduces codebase search entropy. Layer 2 reveals that this result is incomplete if treated as the entire story.

Taken together, the two layers imply that plans provide **two distinct signals**:

1. **Navigation signal**: a file list that narrows what the agent should read.
2. **Reasoning signal**: strategic context about task structure, assumptions, risks, and likely implementation approach.

These signals behave differently. The navigation signal is strongest on localized and cross-module tasks and weakens sharply on architectural tasks. The reasoning signal is more robust and often remains useful even when navigation quality is poor.

This resolves an apparent contradiction in the results. Plans help file targeting, yet hard-constraining code generation to plans hurts. The reconciliation is that plans should be consumed differently at different stages:

> **Supported pipeline:** issue → plan → read planned files → code from file contents
> **Not supported:** issue → plan → generate code directly from the plan

7 Implications for Agent Design

The findings suggest several concrete design recommendations for systems like Devin.

7.1 When to Plan

- **Plan aggressively for localized and cross-module tasks** (2–6 files), where the return on planning is high.
- **Use planning as a scaffold for architectural tasks**, but expect dynamic file discovery during execution.
- **Skip planning for trivially localized tasks** where the affected file is explicit in the issue.

7.2 How to Consume Plans

- Treat **file lists as soft suggestions**, not hard constraints.
- Preserve **reasoning, assumptions, and risks** as durable context even when file targeting is suspect.
- Use plans primarily to decide **what to read**, not **what to write**.

7.3 What a Plan Critic Should Check

The failure analyses and file-difficulty results suggest four concrete critic checks:

1. **Scope plausibility**: does the issue imply a codebase-wide pattern?
2. **Hub module check**: are shared utilities likely involved?
3. **Abstraction-level alignment**: is the plan targeting the right conceptual layer?
4. **Downstream propagation**: is behavior likely to surface in modules that are not statically nearby?

8 Limitations

This project has several important limitations.

- **Bug-fix-heavy task distribution.** The main benchmark is SWE-bench, which emphasizes bug fixing in relatively well-maintained repositories. The synthetic feature-task supplement is small and should be treated only as an external-validity check.
- **Blind code generation.** The code-generation probes did not provide source file contents. Real agents read code before editing it, so the plan-vs-no-plan generation results should be interpreted as a stress test of anchoring, not as a full pipeline evaluation.
- **LLM-as-judge dependence.** Content-quality evaluation used an LLM judge over a small pilot set. These results are useful diagnostics but should not be treated as final ground truth.
- **Static planning only.** The evaluation focuses on one-shot plans generated before execution. Many of the hardest failure modes would likely require dynamic replanning.

9 Future Work

The highest-priority next step is replacing diff-similarity proxies with **test-suite evaluation**. SWE-bench already provides a Docker-based harness and FAIL_TO_PASS tests, and a full test-based evaluation would provide a much stronger answer to whether planning improves downstream patch correctness.

Beyond that, the natural extensions are:

- **Adaptive planning policies** that predict when planning ROI justifies its cost.
- **Dynamic replanning** during execution to address structural ceilings on distant modules.
- **Model cascades** such as Haiku for routine planning and Opus for critic-triggered difficult cases.
- **Richer generalization checks** beyond bug-fix tasks, including larger feature-oriented benchmarks.

10 Conclusion

I designed a two-layer evaluation for planning in coding agents centered on the cheapest grounded proxy for plan usefulness: file targeting. At scale, planning substantially improves navigation quality, especially on localized and cross-module tasks. But the targeted diagnostic probes show that file targeting is not the whole story: navigation accuracy and reasoning quality are only partially coupled.

The main lesson is that planning is not a single capability. It provides both navigation and reasoning, and these should be evaluated—and consumed by agents—differently. Static plans are most useful as *soft scaffolds*: they help agents decide what to read, while their reasoning provides strategic context even when file targeting fails. That reframes the practical question from "should agents plan?" to "which parts of the plan should the agent trust, and at what stage of the pipeline?"